

ΑΡΙΣΤΟΤΕΛΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΟΝΙΚΗΣ
ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ
ΤΜΗΜΑ ΜΑΘΗΜΑΤΙΚΩΝ



Numerical methods for solving systems of chaotic first-order differential equations

ΦΛΩΡΙΑΣ ΠΑΠΑΔΟΠΟΥΛΟΣ

$\sigma = 10$
$\rho = 28,,$
$\beta = 2.667$

Chaotic Lorenz system

$$\begin{cases} x'(t) = \sigma\big(y(t) - x(t)\big) \\ y'(t) = x(t)\big(\rho - z(t)\big) - y(t) \\ z'(t) = x(t)y(t) - \beta z(t) \end{cases}$$

$a = 10$
$b = 2.667$
$c = 46$
$d = 12,,,,$
$e = 10 \ldots.$

Hyperchaotic 4d Lorenz system

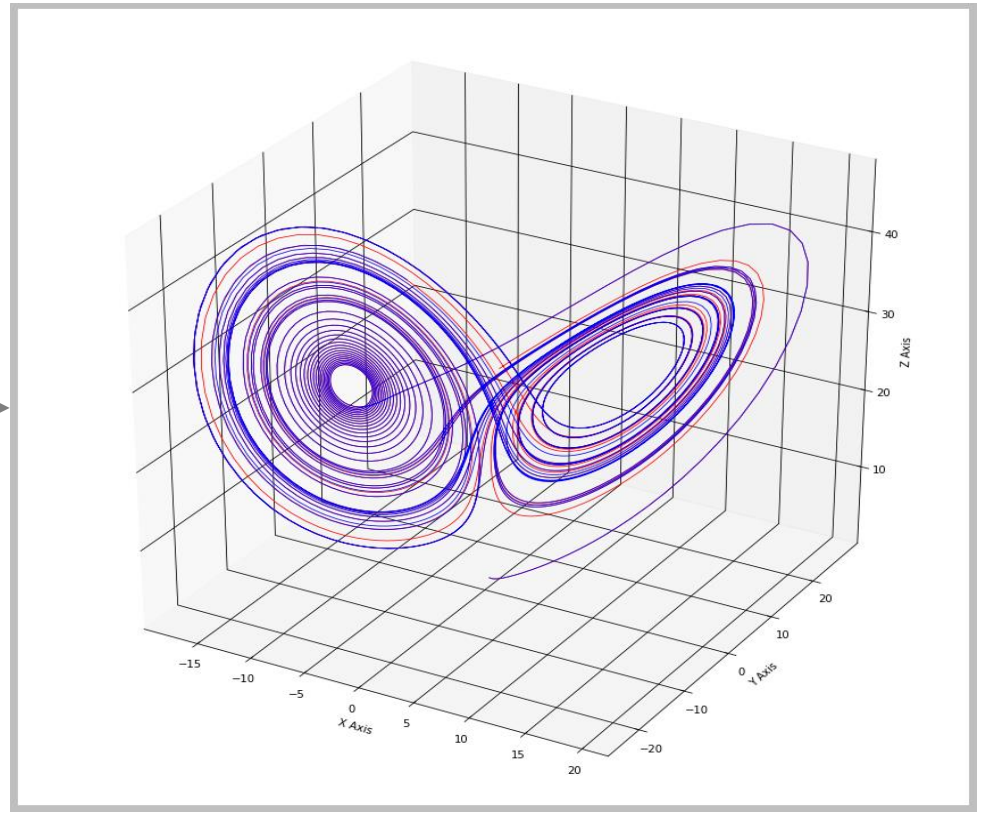
$$\begin{cases} x'(t) = a\big(y(t) - x(t)\big) \\ y'(t) = cx(t) - x(t)z(t) - y(t) + ew(t) \\ z'(t) = x(t)^4 + y(t)^4 - bz(t) \\ w'(t) = -dy(t) \end{cases}$$

$$\begin{cases} x_1{}'(t) = f_1(x_1(t), \ldots, x_n(t)) \\ \vdots \\ x_n{}'(t) = f_n(x_1(t), \ldots, x_n(t)) \end{cases}$$

What does chaotic mean?

*"**Chaos** is aperiodic long-term behavior in a deterministic system that exhibits sensitive dependence on initial conditions"*

- Aperiodic long-term behavior
- Deterministic
- Sensitive dependence on initial conditions



How to solve the generalized system?

Numerical Methods	Explicit Methods	Forward Euler	$X_{k+1} = X_k + h\, f(X_k^T)$
		4th Runge-Kutta	$X_{k+1} = X_k + {}^{h}/_{6}\,[K_1 + 2K_2 + 2K_3 + K_4]$
	Implicit Methods	Backward Euler	Raphson-Newton
		$X_{k+1} = X_k + h\, f(X_{k+1}^T)$	

Approaching the problem with Python

$$\begin{cases} x_1'(t) = f_1(x_1(t), \dots, x_n(t)) \\ \vdots \\ x_n'(t) = f_n(x_1(t), \dots, x_n(t)) \end{cases} \quad \text{or} \quad X' = f$$

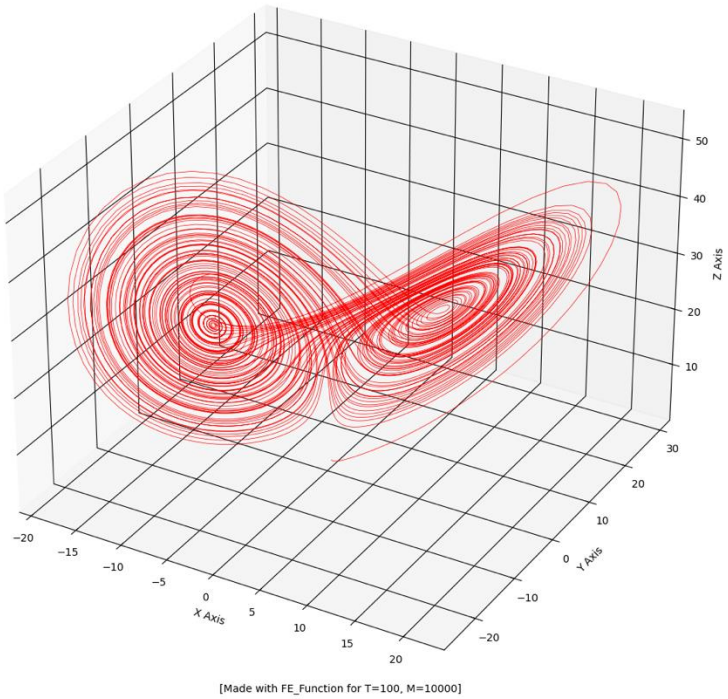
- Creation of $(x_1, \dots, x_n)$ using $SymPy$.
- Definition of $f_1, \dots, f_n$ using $SymPy$.
- Computation of f using $SymPy$.
- Creation of a function for the repeating algorithm.
- Use of the previous function in each iteration of the discretization.
- Output of the solutions $(x_1, \dots, x_n)$ for each iteration.

$$X_{k+1} = X_k + h\, f(X_k^T)$$

+ One extra step in backward Euler is the use of the Newton method.

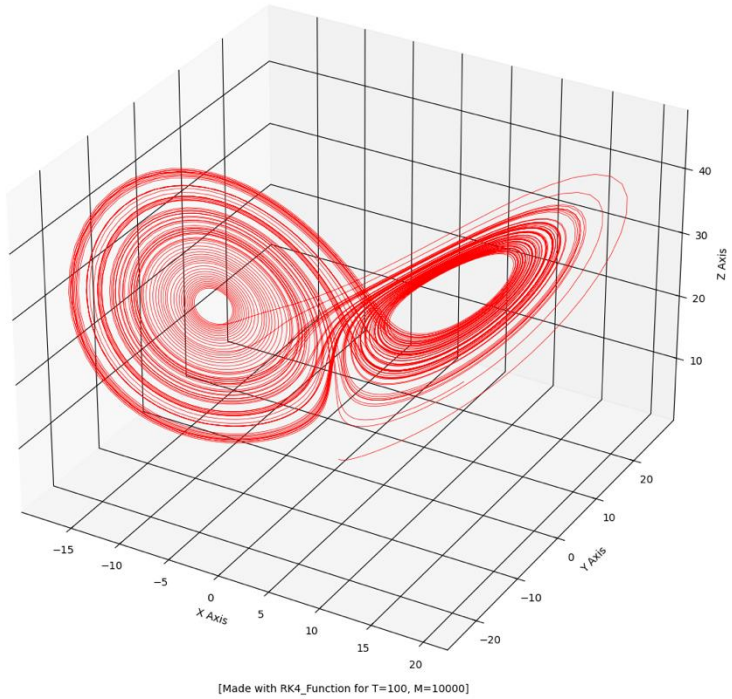

Lorenz
systems

Lorenz system



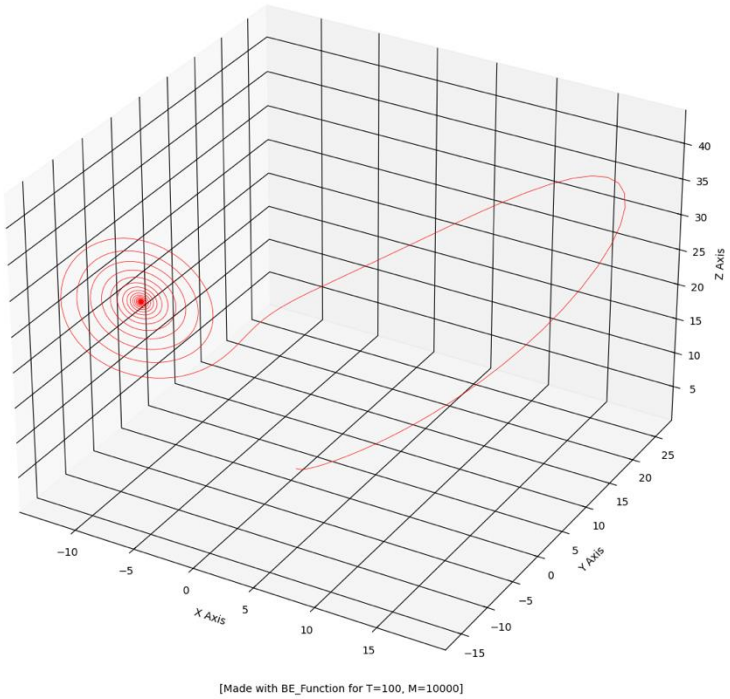
Forward Euler

h=0.01



4th order Runge-Kutta

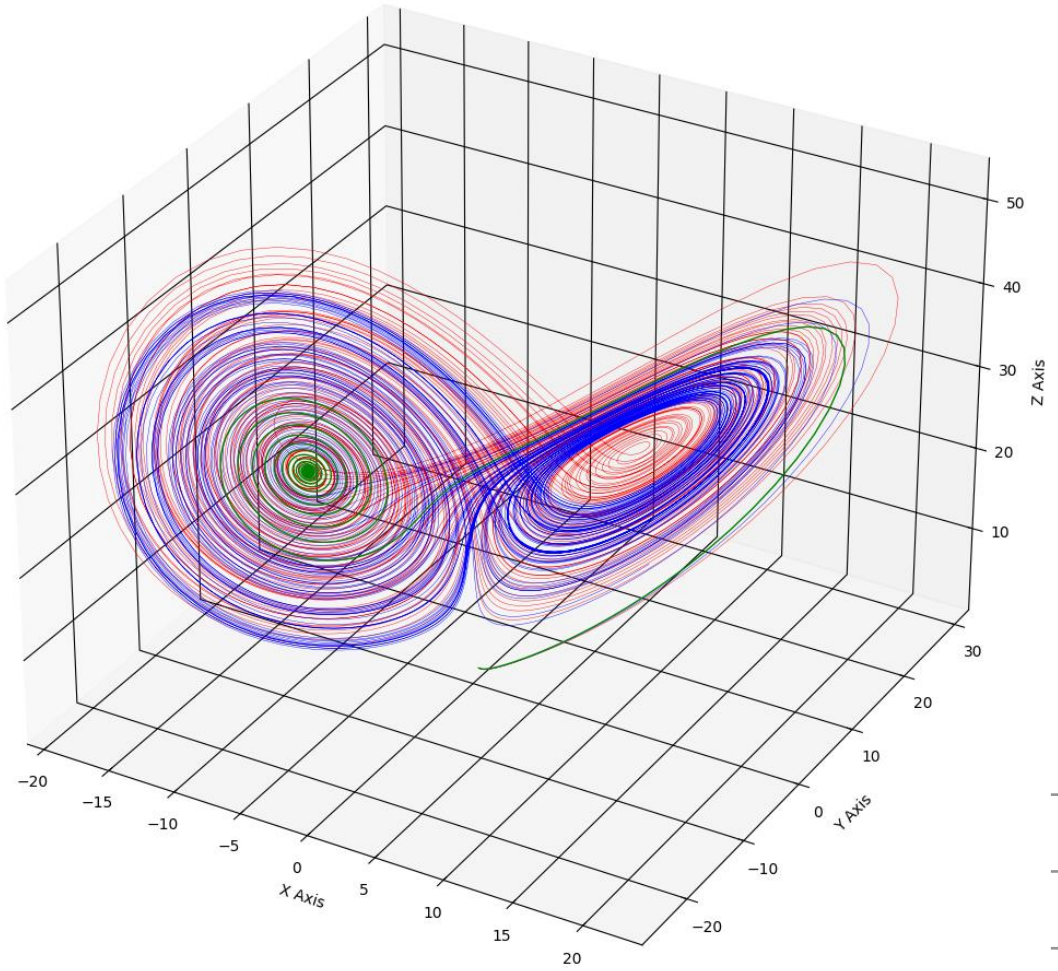
h=0.01



Backward Euler (with Newton)

h=0.01

Lorenz system



[Made with FE_Function, RK4_Function and BE_Function for T=100, M=10000]

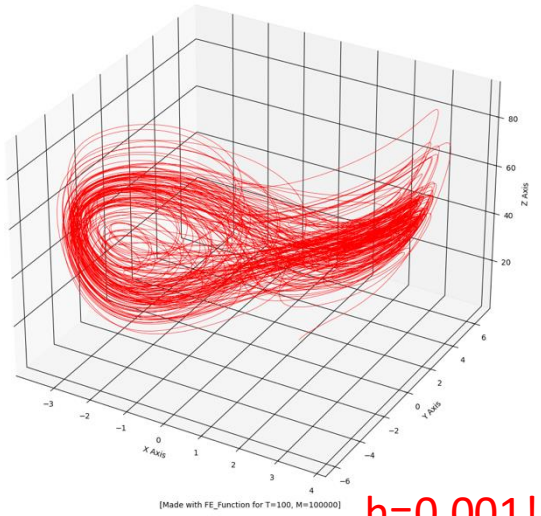
Forward Euler

4th order Runge-Kutta

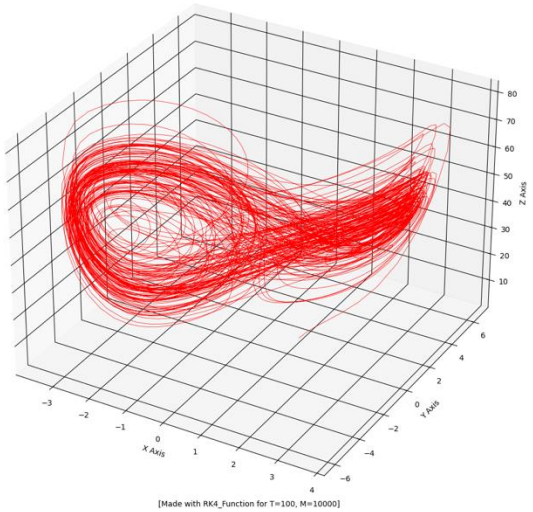
Backward Euler (with Newton)

4d Lorenz system

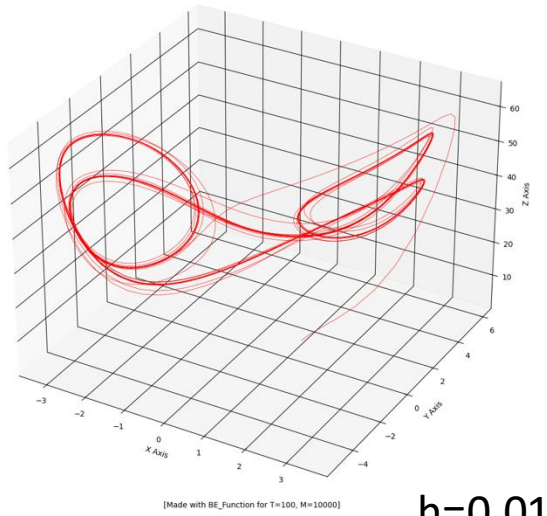
xyz:



h=0.001!

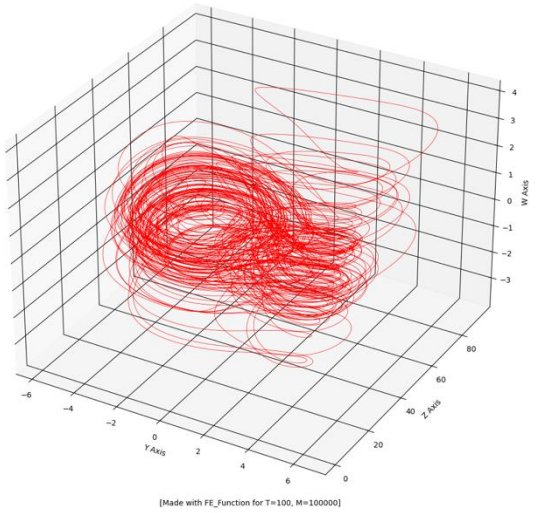


h=0.01

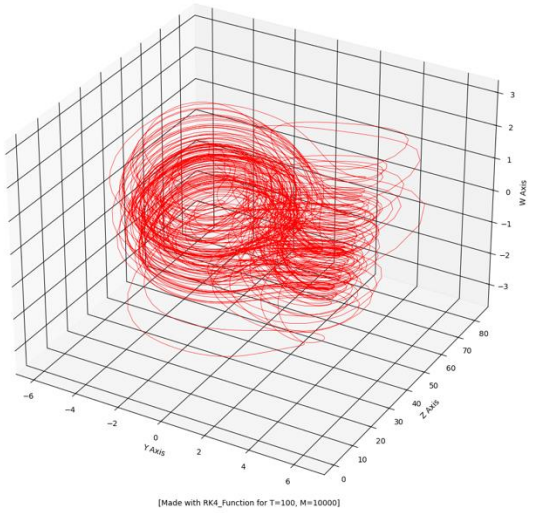


h=0.01

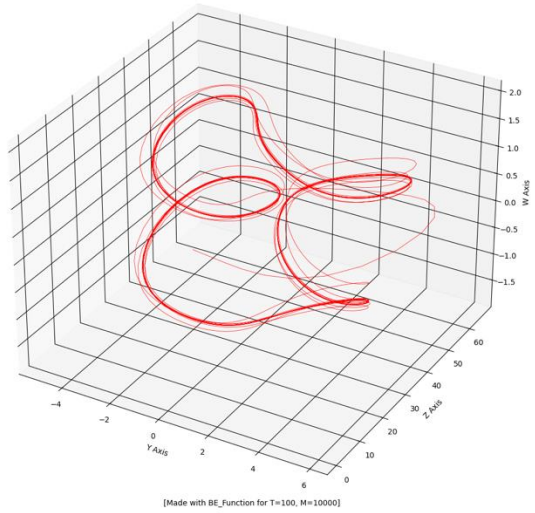
yzw:



Forward Euler

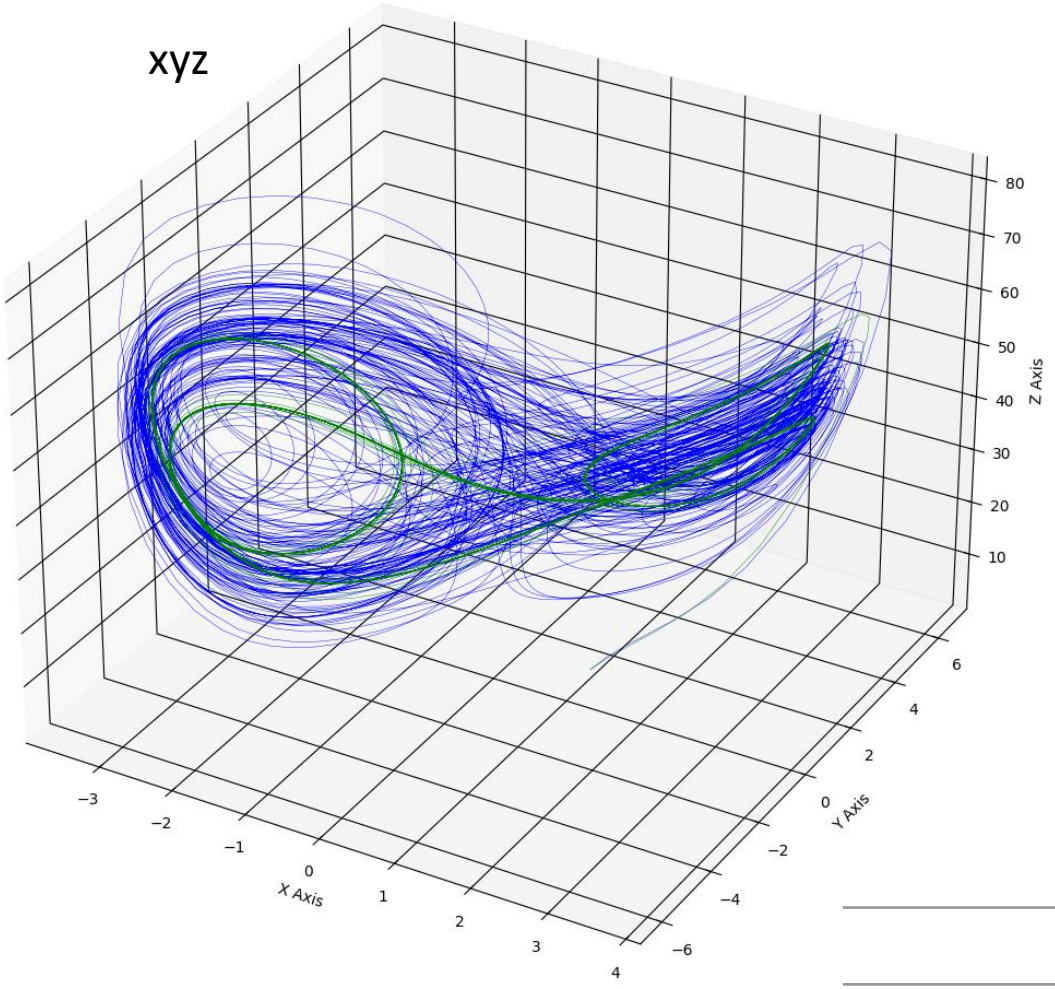


4th order Runge-Kutta



Backward Euler (with Newton)

4d Lorenz system

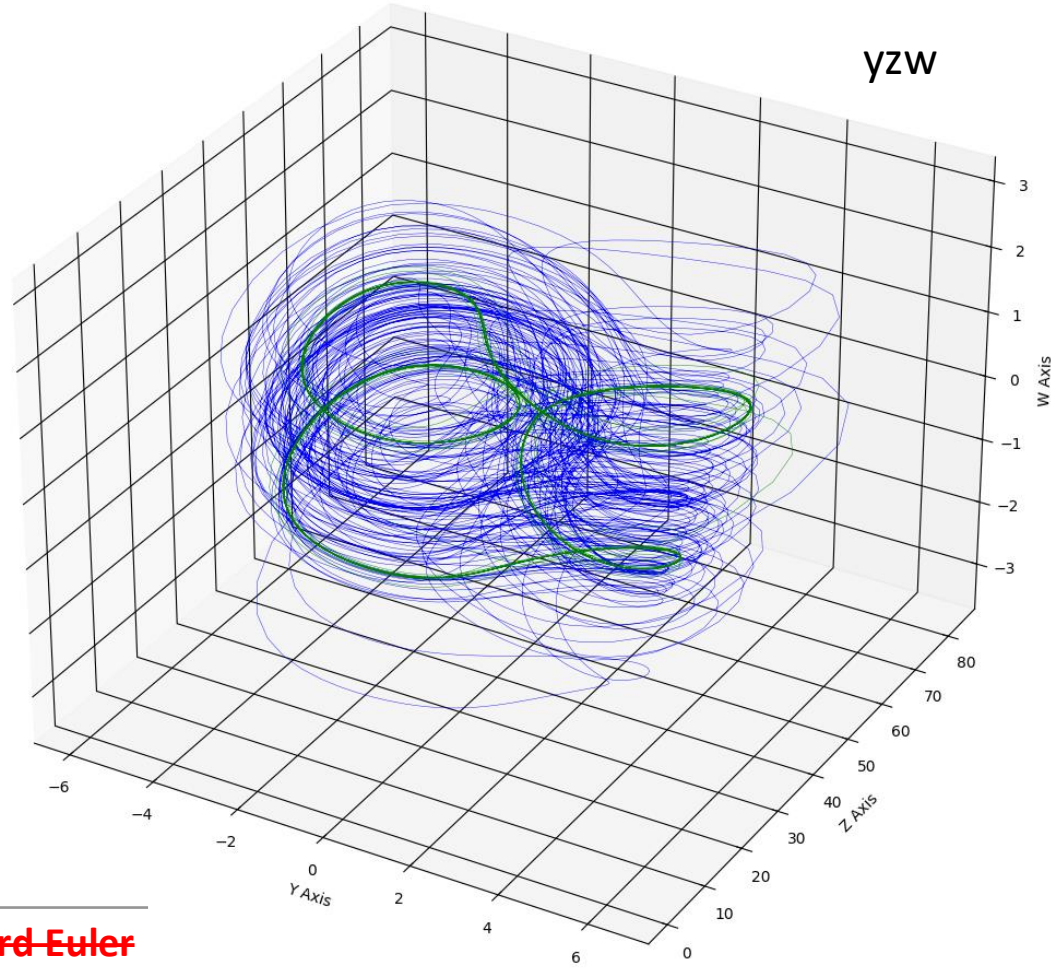


[Made with RK4_Function and BE_Function for T=100, M=10000]

~~Forward Euler~~

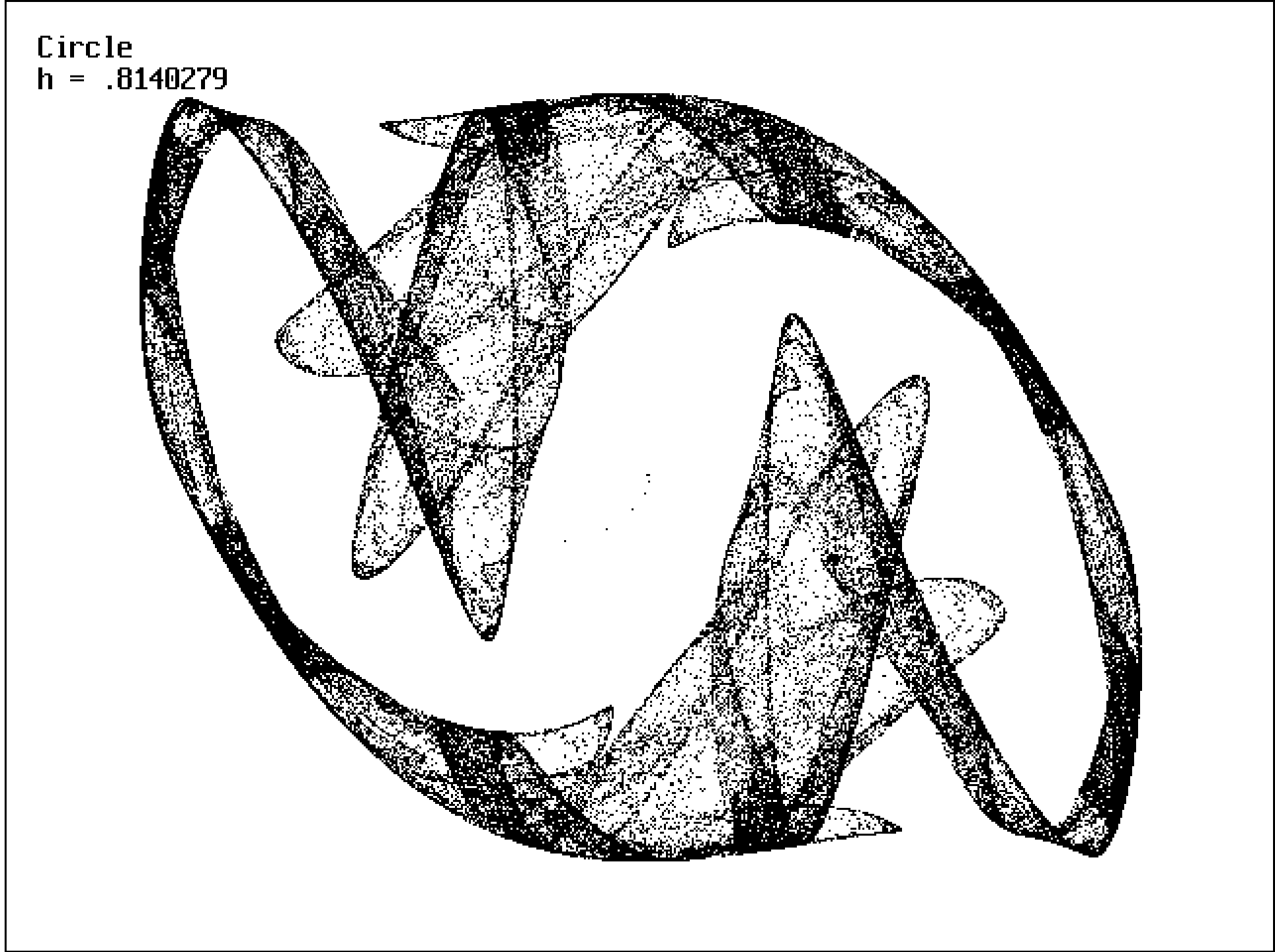
4th order Runge-Kutta

Backward Euler (with Newton)



[Made with RK4_Function and BE_Function for T=100, M=10000]

Descending further into chaos

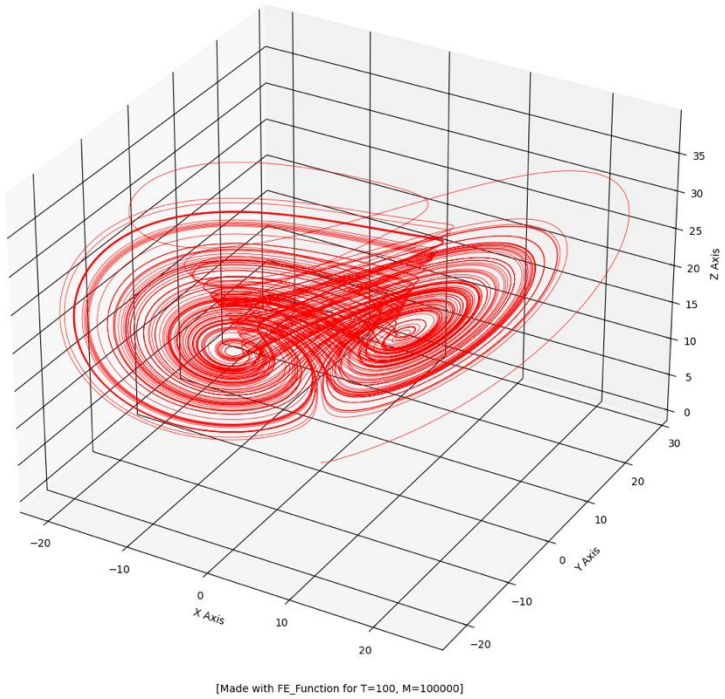


Chen system

$$\begin{cases} x'(t) = a\big(y(t) - x(t)\big) \\ y'(t) = (c-a)x(t) - x(t)z(t) + cy(t) \\ z'(t) = x(t)y(t) - bz(t) \end{cases}$$

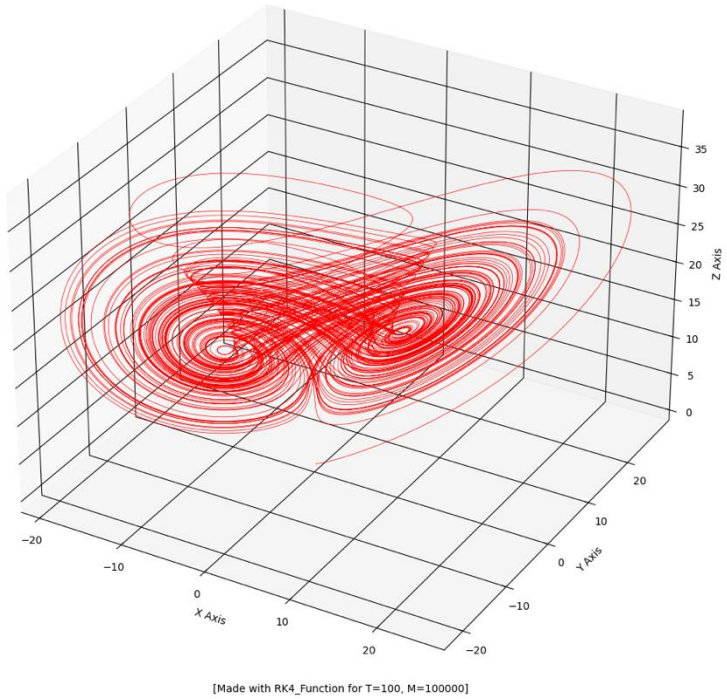
$a = 40$
$b = 3$
$c = 28$

Chen system



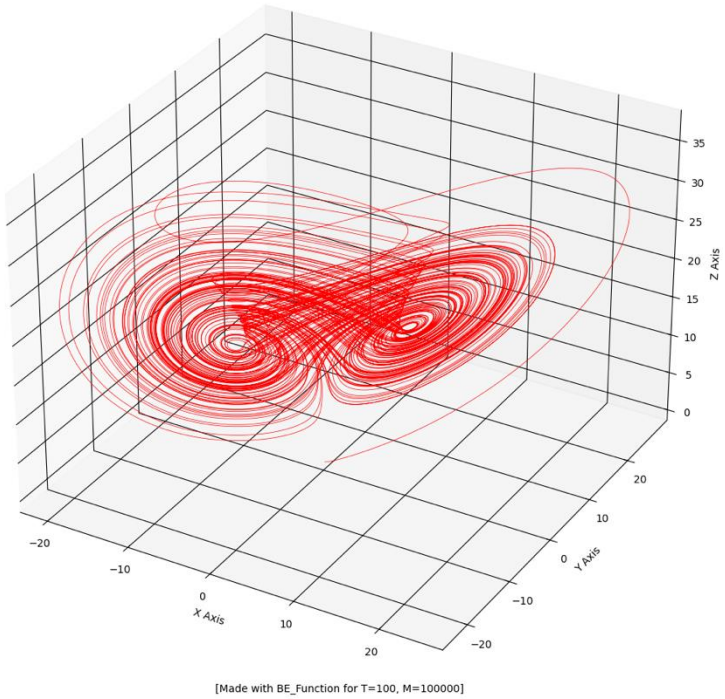
Forward Euler

h=0.001



4th order Runge-Kutta

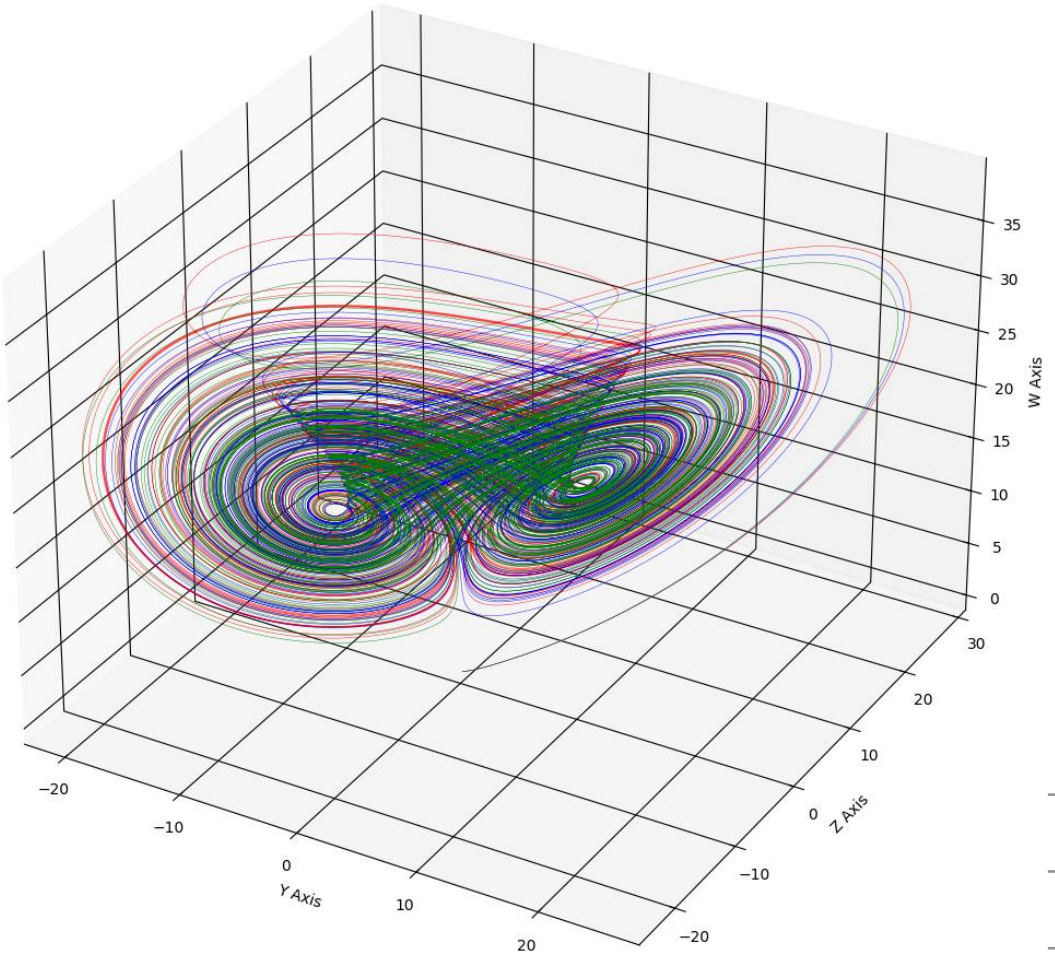
h=0.001



Backward Euler (with Newton)

h=0.001

Chen system



[Made with FE_function, RK4_Function and BE_Function for T=100, M=100000]

Forward Euler

4th order Runge-Kutta

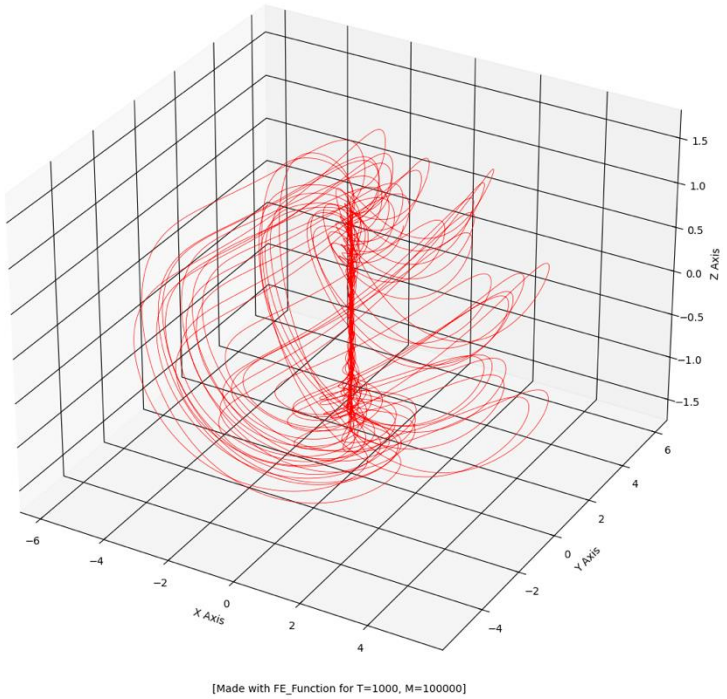
Backward Euler (with Newton)

Nose-Hoover Oscillator

$$\begin{cases} x'(t) = y(t) \\ y'(t) = -x(t) - y(t)z(t) \\ z'(t) = {(y(t)^2 - 1)}/{q} \end{cases}$$

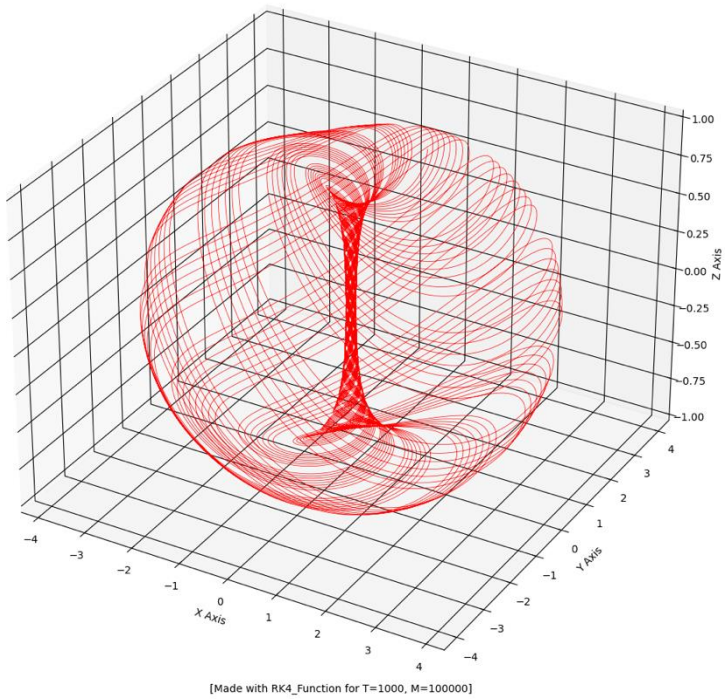
$$q = 10$$

Nose-Hoover Oscillator



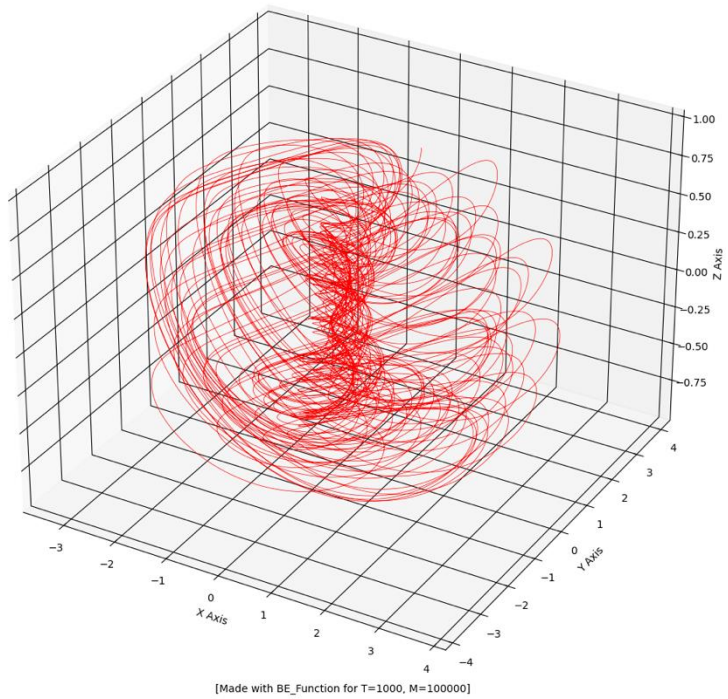
Forward Euler

h=0.01



4th order Runge-Kutta

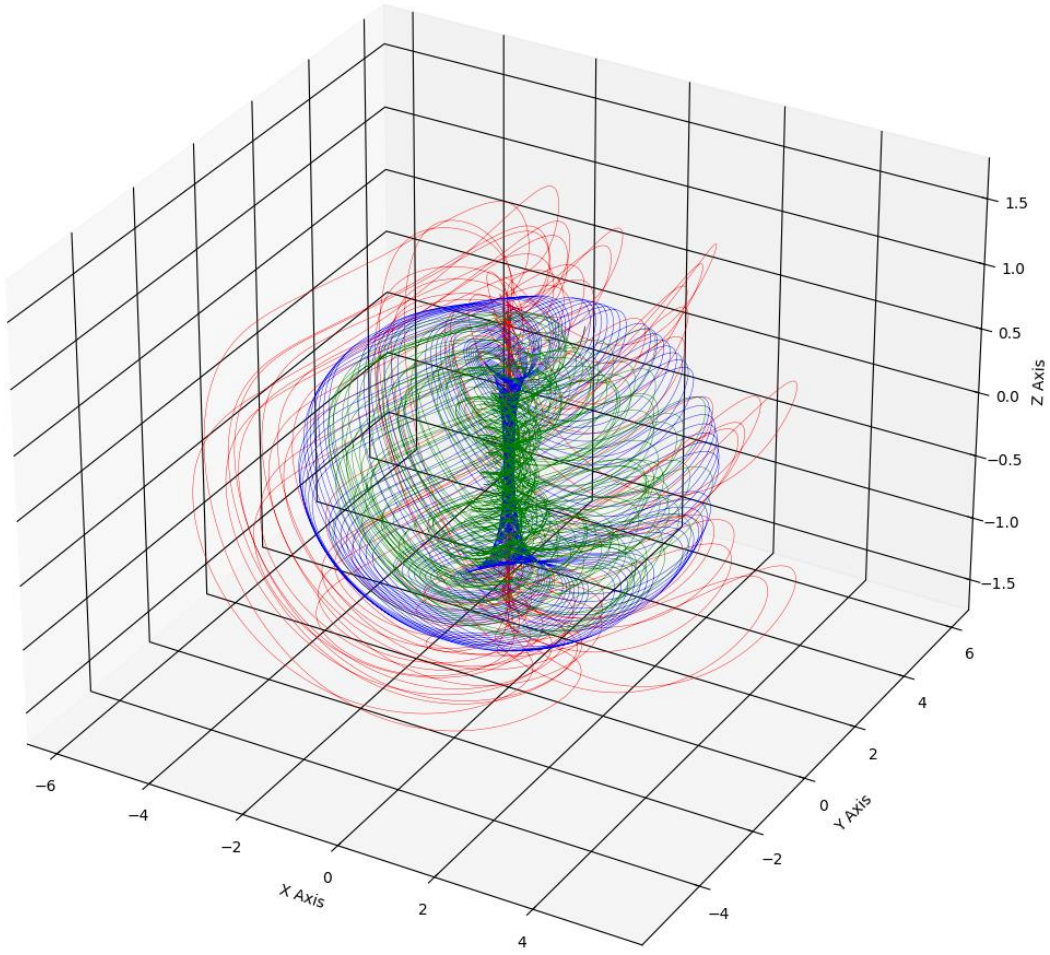
h=0.01



Backward Euler (with Newton)

h=0.01

Nose-Hoover Oscillator



[Made with FE_Function, RK4_Function and BE_Function for T=1000, M=100000]

Forward Euler

4th order Runge-Kutta

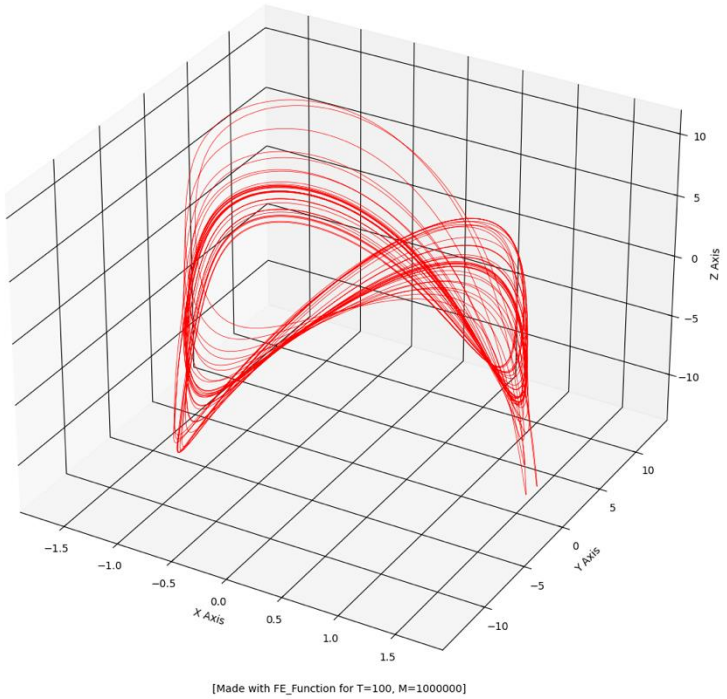
Backward Euler (with Newton)

Sprott-Jafari system

$$\begin{cases} x'(t) = y(t) \\ y'(t) = -x(t) + y(t)z(t) \\ z'(t) = z(t) + ax(t)^2 - y(t)^2 - b \end{cases}$$

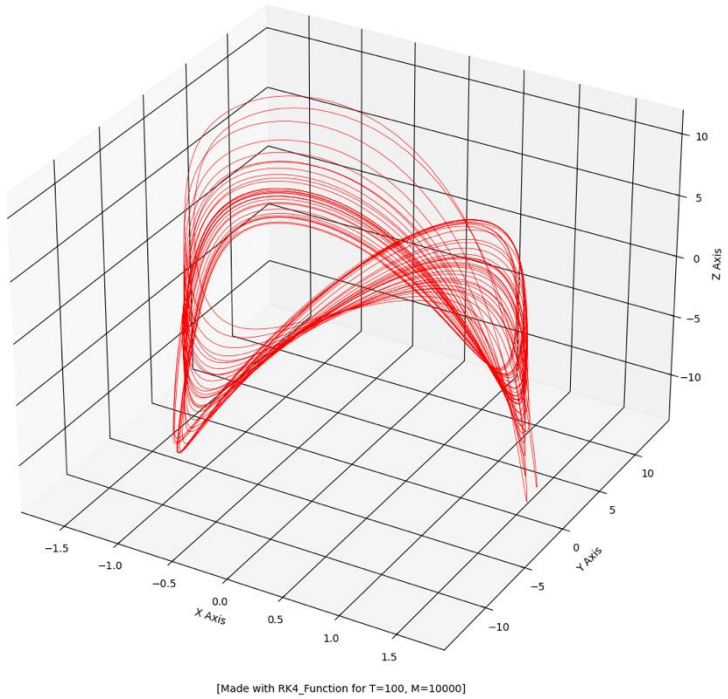
$$a = 8.894$$
$$b = 4$$

Sprott-Jafari system



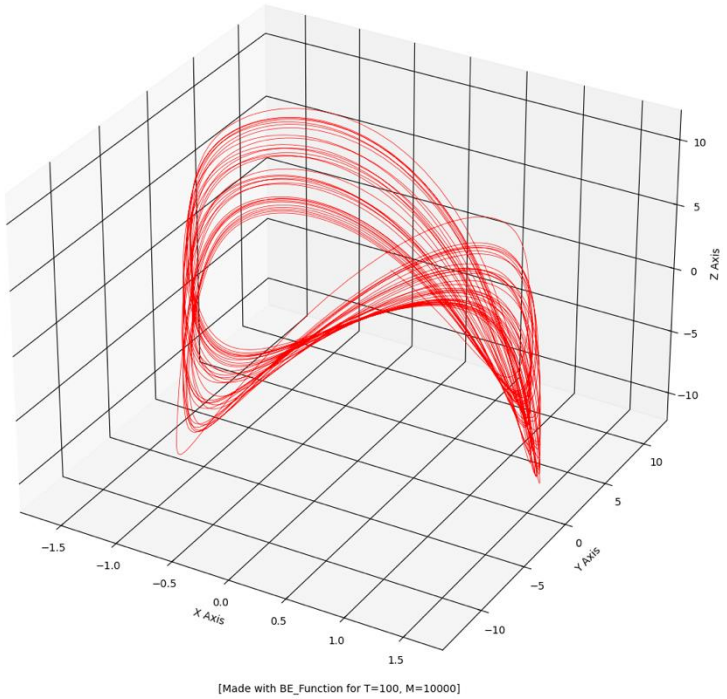
Forward Euler

h=0.0001!



4th order Runge-Kutta

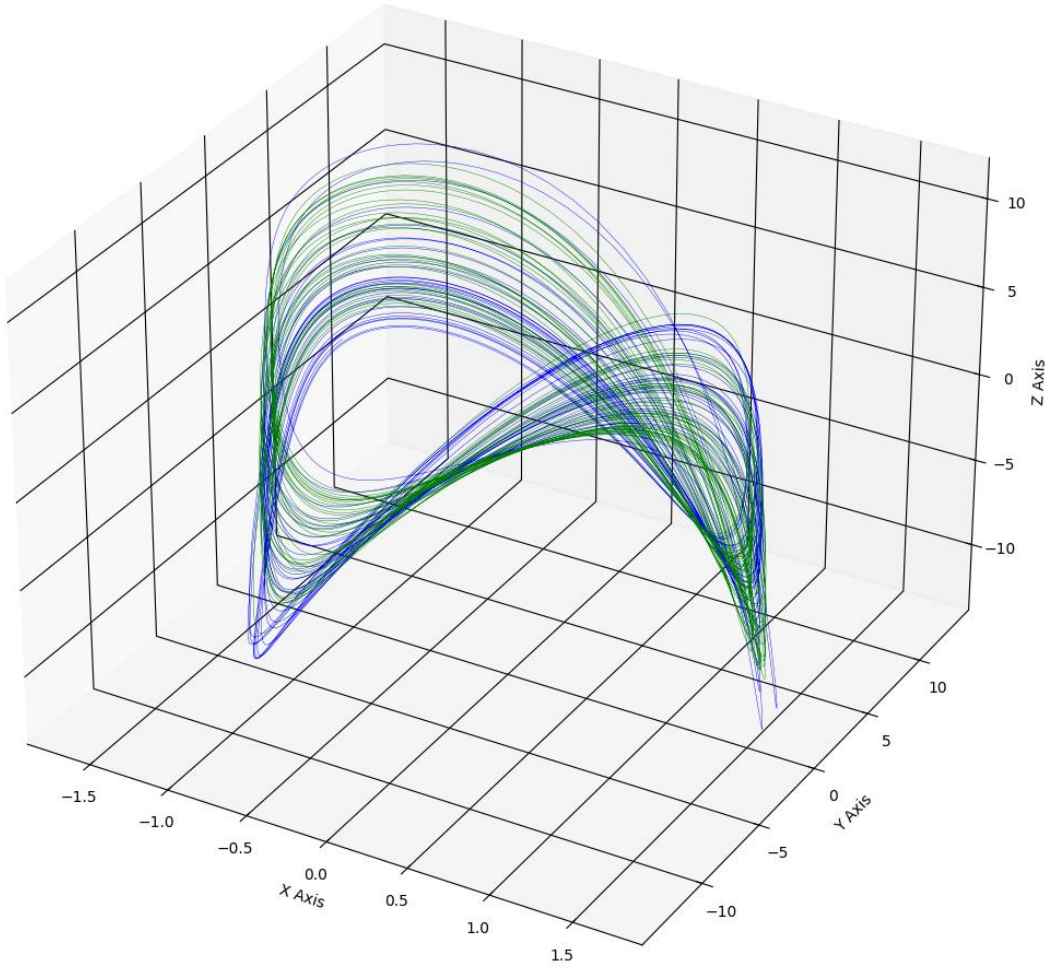
h=0.01



Backward Euler (with Newton)

h=0.01

Sprott-Jafari system



[Made with RK4_Function and BE_Function for T=100, M=10000]

~~Forward Euler~~

4th order Runge-Kutta

Backward Euler (with Newton)

Animation

- File: chaotic_presentation.ipynb